

# Lesson 2 : Advanced SQL Queries

Prepared By: Ved Vyas

In SQL, **AND** and **OR** are logical operators used in the **WHERE** clause to filter rows.

### **AND** Operator

Returns rows **only if all conditions are true**.

### **OR** Operator

Returns rows **if at least one condition is true**.

| Operator | Meaning                             | Example                        |
|----------|-------------------------------------|--------------------------------|
| AND      | All conditions must be true         | Age > 18 AND City = 'New York' |
| OR       | At least one condition must be true | Age > 18 OR City = 'New York'  |

Table: Customers

| customer_id | first_name | last_name | age | country |
|-------------|------------|-----------|-----|---------|
| 1           | John       | Doe       | 31  | USA     |
| 2           | Robert     | Luna      | 22  | USA     |
| 3           | David      | Robinson  | 22  | UK      |
| 4           | John       | Reinhardt | 25  | UK      |
| 5           | Betty      | Doe       | 28  | UAE     |



```
SELECT first_name, last_name  
FROM Customers  
WHERE country = 'USA' OR last_name = 'Doe';
```



| first_name | last_name |
|------------|-----------|
| John       | Doe       |
| Robert     | Luna      |
| Betty      | Doe       |

```
CREATE TABLE Players (  
    PlayerID INT,  
    Name VARCHAR(50),  
    Sport VARCHAR(30),  
    Age INT,  
    City VARCHAR(30),  
    Team VARCHAR(30)  
);
```

```
INSERT INTO Players VALUES (1, 'Virat', 'Cricket', 35, 'Delhi', 'India');  
INSERT INTO Players VALUES (2, 'Rohit', 'Cricket', 37, 'Mumbai', 'India');  
INSERT INTO Players VALUES (3, 'Neeraj', 'Athletics', 28, 'Chandigarh', 'India');  
INSERT INTO Players VALUES (4, 'Sania', 'Tennis', 30, 'Hyderabad', 'India');  
INSERT INTO Players VALUES (5, 'Sunil', 'Football', 29, 'Goa', 'India');  
INSERT INTO Players VALUES (6, 'PV Sindhu', 'Badminton', 31, 'Hyderabad', 'India');
```

| PlayerID | Name      | Sport     | Age | City       | Team  |
|----------|-----------|-----------|-----|------------|-------|
| 1        | Virat     | Cricket   | 35  | Delhi      | India |
| 2        | Rohit     | Cricket   | 37  | Mumbai     | India |
| 3        | Neeraj    | Athletics | 28  | Chandigarh | India |
| 4        | Sania     | Tennis    | 30  | Hyderabad  | India |
| 5        | Sunil     | Football  | 29  | Goa        | India |
| 6        | PV Sindhu | Badminton | 31  | Hyderabad  | India |

Find players who play **Cricket** AND are older than 32.

**SELECT \* FROM Players WHERE Sport = 'Cricket' AND Age > 32;**

| PlayerID | Name  | Sport   | Age | City   | Team  |
|----------|-------|---------|-----|--------|-------|
| 1        | Virat | Cricket | 35  | Delhi  | India |
| 2        | Rohit | Cricket | 37  | Mumbai | India |

Find players who play **Cricket** OR **Tennis**.

**SELECT \*  
FROM Players  
WHERE Sport = 'Cricket'  
OR Sport = 'Tennis';**

| PlayerID | Name  | Sport   | Age | City      | Team  |
|----------|-------|---------|-----|-----------|-------|
| 1        | Virat | Cricket | 35  | Delhi     | India |
| 2        | Rohit | Cricket | 37  | Mumbai    | India |
| 4        | Sania | Tennis  | 30  | Hyderabad | India |

## OR with different conditions

Find players who are from **Delhi** OR play **Football**.

 SQL

```
SELECT *  
FROM Players  
WHERE City = 'Delhi'  
OR Sport = 'Football';
```

### Result:

| Name  | Sport    | City  |
|-------|----------|-------|
| Virat | Cricket  | Delhi |
| Sunil | Football | Goa   |

Find players who are from Hyderabad AND play Tennis or Badminton.

 SQL



```
SELECT *  
FROM Players  
WHERE City = 'Hyderabad'  
AND (Sport = 'Tennis' OR Sport = 'Badminton');
```

**Result:**

| Name      | Sport     | City      |
|-----------|-----------|-----------|
| Sania     | Tennis    | Hyderabad |
| PV Sindhu | Badminton | Hyderabad |



Find players who play Cricket or Football AND are older than 30.

 SQL



```
SELECT *  
FROM Players  
WHERE (Sport = 'Cricket' OR Sport = 'Football')  
AND Age > 30;
```

| PlayerID | Name  | Sport   | Age | City   | Team  |
|----------|-------|---------|-----|--------|-------|
| 1        | Virat | Cricket | 35  | Delhi  | India |
| 2        | Rohit | Cricket | 37  | Mumbai | India |

```
1 CREATE TABLE exercise_logs
2     (id INTEGER PRIMARY KEY AUTOINCREMENT,
3      type TEXT,
4      minutes INTEGER,
5      calories INTEGER,
6      heart_rate INTEGER);
7
8
9 INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
  ("biking", 30, 100, 110);
10 INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
  ("biking", 10, 30, 105);
11 INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
  ("dancing", 15, 200, 120);
12
13 SELECT * FROM exercise_logs WHERE calories > 50 ORDER BY calories;
14
15 /* AND */
16 SELECT * FROM exercise_logs WHERE calories > 50 AND minutes < 30;
17
18 /* OR */
19 SELECT * FROM exercise_logs WHERE calories > 50 OR heart_rate > 100;
```

## QUERY RESULTS

| id | type    | minutes | calories | heart_rate |
|----|---------|---------|----------|------------|
| 1  | biking  | 30      | 100      | 110        |
| 3  | dancing | 15      | 200      | 120        |

| id | type    | minutes | calories | heart_rate |
|----|---------|---------|----------|------------|
| 3  | dancing | 15      | 200      | 120        |

| id | type    | minutes | calories | heart_rate |
|----|---------|---------|----------|------------|
| 1  | biking  | 30      | 100      | 110        |
| 2  | biking  | 10      | 30       | 105        |
| 3  | dancing | 15      | 200      | 120        |

# SQL Subquery

A subquery in SQL is a query nested inside another SQL query. It allows complex filtering, aggregation and data manipulation by using the result of one query inside another. They are an essential tool when we need to perform operations like:

- Apply aggregate functions like SUM, COUNT or AVG dynamically.
- Update data using values from other tables.
- Delete rows based on conditions returned by another query.

SQL operators are special symbols or keywords used to perform operations on data stored in a database. They are broadly categorized as follows:

| Operator Type                        | Operators                                                                                                                                        | Purpose                                      |  |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|-------------------------------------------------------------------------------------|
| Arithmetic Operators                 | <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>                                                               | Perform mathematical calculations.           |                                                                                     |
| Comparison<br>(Relational) Operators | <code>=</code> , <code>!=</code> or <code>&lt;&gt;</code> , <code>&gt;</code> , <code>&lt;</code> , <code>&gt;=</code> , <code>&lt;=</code>      | Compare two values and return TRUE or FALSE. |                                                                                     |
| Logical Operators                    | <code>AND</code> , <code>OR</code> , <code>NOT</code>                                                                                            | Combine multiple conditions.                 |                                                                                     |
| Special Operators                    | <code>IN</code> , <code>BETWEEN</code> , <code>LIKE</code> , <code>IS NULL</code> ,<br><code>EXISTS</code> , <code>ANY</code> , <code>ALL</code> | Perform special types of comparisons.        |                                                                                     |
| Set Operators                        | <code>UNION</code> , <code>UNION ALL</code> , <code>INTERSECT</code> ,<br><code>EXCEPT</code> (or <code>MINUS</code> in Oracle)                  | Combine results from multiple queries.       |                                                                                     |

# Syntax of subqueries

<https://www.geeksforgeeks.org/sql/sql-subquery/>

```
SELECT column_name  
FROM table_name  
WHERE column_name operator  
      (SELECT column_name  
        FROM table_name  
        WHERE condition);
```

An **SQL subquery** is a query nested inside another SQL statement (such as **SELECT**, **INSERT**, **UPDATE**, or **DELETE**). The nested query is called the **inner query**, while the surrounding statement is called the **outer query**.

# Example: Teacher Table

```
CREATE TABLE Teacher (
```

```
    teacher_id INT,
```

```
    teacher_name VARCHAR(50),
```

```
    subject VARCHAR(50),
```

```
    salary INT,
```

```
    experience INT,
```

```
    department VARCHAR(50)
```

```
);
```

```
INSERT INTO Teacher VALUES (1, 'Amit', 'Maths', 50000, 5, 'Science');
```

```
INSERT INTO Teacher VALUES (2, 'Priya', 'Physics', 60000, 8, 'Science');
```

```
INSERT INTO Teacher VALUES (3, 'Rahul', 'English', 45000, 3, 'Arts');
```

```
INSERT INTO Teacher VALUES (4, 'Neha', 'Chemistry', 55000, 6, 'Science');
```

```
INSERT INTO Teacher VALUES (5, 'Karan', 'History', 40000, 2, 'Arts');
```

| teacher_id | teacher_name | subject   | salary | experience | department |  |
|------------|--------------|-----------|--------|------------|------------|-------------------------------------------------------------------------------------|
| 1          | Amit         | Maths     | 50000  | 5          | Science    |                                                                                     |
| 2          | Priya        | Physics   | 60000  | 8          | Science    |                                                                                     |
| 3          | Rahul        | English   | 45000  | 3          | Arts       |                                                                                     |
| 4          | Neha         | Chemistry | 55000  | 6          | Science    |                                                                                     |
| 5          | Karan        | History   | 40000  | 2          | Arts       |                                                                                     |



Write a query to display the teachers whose salary is greater than the average salary of all teachers.

```
SELECT teacher_name,  
salary  
FROM Teacher  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM Teacher  
);
```

Result:

teacher\_name

salary

Priya

60000

Neha

55000

**Ques:** Write a query to display all teachers whose salary is less than the highest salary.

# Understand the execution...

First, the subquery runs:

```
SELECT AVG(salary) FROM Teacher;
```

The average salary is:

```
(50000 + 60000 + 45000 + 55000 + 40000) / 5  
= 50000
```

So the outer query becomes conceptually:

```
SELECT teacher_name, salary  
FROM Teacher  
WHERE salary > 50000;
```

Write a query to display teachers who have more experience than Amit.

```
SELECT teacher_name, experience
FROM Teacher
WHERE experience > (
    SELECT experience
    FROM Teacher
    WHERE teacher_name = 'Amit'
);
```

| teacher_name | experience |  |
|--------------|------------|-------------------------------------------------------------------------------------|
| Priya        | 8          |                                                                                     |
| Neha         | 6          |                                                                                     |

Que. Write a query to display teachers whose salary is greater than Amit's salary.

Write a query to display all teachers who belong to the same department as Priya.

```
SELECT teacher_name, department
FROM Teacher
WHERE department = (
    SELECT department
    FROM Teacher
    WHERE teacher_name = 'Priya'
);
```

| teacher_name | department |
|--------------|------------|
| Amit         | Science    |
| Priya        | Science    |
| Neha         | Science    |

**subquery in the FROM clause.** Here, the result of the inner query is treated like a temporary table.

question) Display the teachers whose salary is greater than the average salary.

```
SELECT teacher_name, salary
FROM (
    SELECT teacher_name, salary
    FROM Teacher
    WHERE salary > 50000
) AS T;
```

**Inner query:**

```
SELECT teacher_name, salary
FROM Teacher
WHERE salary > 50000;
```

The result is treated as a temporary table named T.

**Outer query:**

```
SELECT teacher_name, salary FROM T;
```

### 3. Subquery in `SELECT`

A subquery can also be used to calculate a value that appears as a column.

**Example:** Display every teacher along with the average salary.

`</>` SQL



```
SELECT teacher_name,  
       salary,  
       (SELECT AVG(salary) FROM Teacher) AS average_salary  
FROM Teacher;
```

| teacher_name | salary | average_salary |  |
|--------------|--------|----------------|-------------------------------------------------------------------------------------|
| Amit         | 50000  | 50000          |                                                                                     |
| Priya        | 60000  | 50000          |                                                                                     |
| Rahul        | 45000  | 50000          |                                                                                     |
| Neha         | 55000  | 50000          |                                                                                     |
| Karan        | 40000  | 50000          |                                                                                     |

In SQL, a **subquery** can be written in different parts of a **query**, depending on what you want to achieve.

## Subquery with Two Tables in SQL

When working with **two tables**, a subquery is commonly used to retrieve values from one table and use those values to filter or compare data in another table.



```
1 CREATE TABLE exercise_logs
2     (id INTEGER PRIMARY KEY AUTOINCREMENT,
3      type TEXT,
4      minutes INTEGER,
5      calories INTEGER,
6      heart_rate INTEGER);
7
8 INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
9 ("biking", 30, 100, 110);
10 INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
11 ("biking", 10, 30, 105);
12 INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
13 ("dancing", 15, 200, 120);
14 INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
15 ("tree climbing", 30, 70, 90);
16 INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
17 ("tree climbing", 25, 72, 80);
18 INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
19 ("rowing", 30, 70, 90);
20 INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
21 ("hiking", 60, 80, 85);
22
23 SELECT * FROM exercise_logs;
```

## QUERY RESULTS

| id | type          | minutes | calories | heart_rate |
|----|---------------|---------|----------|------------|
| 1  | biking        | 30      | 100      | 110        |
| 2  | biking        | 10      | 30       | 105        |
| 3  | dancing       | 15      | 200      | 120        |
| 4  | tree climbing | 30      | 70       | 90         |
| 5  | tree climbing | 25      | 72       | 80         |
| 6  | rowing        | 30      | 70       | 90         |
| 7  | hiking        | 60      | 80       | 85         |

## SQL GROUP BY with HAVING — Detailed Explanation

GROUP BY and HAVING are commonly used together when we want to group rows, perform aggregate calculations on each group, and then filter those groups.

The easiest way to understand them is:

GROUP BY creates groups. HAVING filters those groups.

# AGGREGATION FUNCTIONS

| Function | Purpose             |
|----------|---------------------|
| COUNT()  | Counts rows         |
| SUM()    | Calculates total    |
| AVG()    | Calculates average  |
| MAX()    | Finds maximum value |
| MIN()    | Finds minimum value |



**GROUP BY** is used to **combine rows having the same value into groups**.

employees

| id | name   | department | salary |
|----|--------|------------|--------|
| 1  | Rahul  | IT         | 50000  |
| 2  | Priya  | HR         | 40000  |
| 3  | Amit   | IT         | 60000  |
| 4  | Neha   | HR         | 45000  |
| 5  | Raj    | Sales      | 55000  |
| 6  | Anjali | IT         | 70000  |
| 7  | Karan  | Sales      | 50000  |

If we want to find the **total salary of each department**, we use:

`</>` SQL



```
SELECT department, SUM(salary)
FROM employees
GROUP BY department;
```

## Result

| department | SUM(salary) |
|------------|-------------|
| IT         | 180000      |
| HR         | 85000       |
| Sales      | 105000      |

## 4. GROUP BY with COUNT()

Suppose we want to know how many employees are in each department.

 SQL



```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department;
```

### Result

| department | COUNT(*) |
|------------|----------|
| IT         | 3        |
| HR         | 2        |
| Sales      | 2        |

## 5. GROUP BY with AVG()

To find the average salary of each department:

 SQL



```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

### Result

| department | AVG(salary) |
|------------|-------------|
| IT         | 60000       |
| HR         | 42500       |
| Sales      | 52500       |



## 6. GROUP BY with MAX() and MIN()

We can also find the highest and lowest salary in every department.

 SQL 

```
SELECT department,  
       MAX(salary) AS highest_salary,  
       MIN(salary) AS lowest_salary  
FROM employees  
GROUP BY department;
```

### Result

| department | highest_salary | lowest_salary |
|------------|----------------|---------------|
| IT         | 70000          | 50000         |
| HR         | 45000          | 40000         |
| Sales      | 55000          | 50000         |

# HAVING - APPLY CONDITIONS ON AGGREGATIONS

</> SQL



```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department
HAVING SUM(salary) > 100000;
```

## Result

| department | total_salary |
|------------|--------------|
| IT         | 180000       |
| Sales      | 105000       |